Atividade Integradora - Java e POO

Laboratório de Programação Orientada a Objetos | UniAteneu

Tema: Sistema Acadêmico Simplificado | Aulas 01 a 08

Aluno: Lucas Emanoel da Silva Maia

Parte I - Questões teóricas

Bloco A - Fundamentos de POO

1. Explique a diferença entre classe e objeto. Apresente um exemplo de cada um em um sistema acadêmico.

A classe é como se fosse o molde ou a planta de uma casa: ela diz como as coisas daquele tipo são, quais dados elas têm e o que elas fazem. O objeto é o que a gente cria usando esse molde, ou seja, a casa de verdade, com os valores dele. No sistema acadêmico, `Aluno` é a classe, porque define que todo aluno tem nome e cursos. Já `new Aluno("Ana")` é um objeto, a aluna Ana. Da mesma classe dá para criar vários objetos, cada um com seus próprios dados.

2. O que são atributos e métodos? Diferencie estado e comportamento de um objeto.

Atributos são as variáveis da classe, onde ficam guardados os dados do objeto, como o `nome` do aluno. Métodos são as ações que o objeto sabe fazer, como `adicionarCurso()`. O estado é o conjunto dos valores que os atributos têm naquele momento. O comportamento é o que o objeto faz através dos métodos. Os métodos geralmente usam ou mudam o estado.

3. Qual é a função de new? Diferencie criar um objeto e declarar uma referência.

O `new` serve para criar um objeto na memória e chamar o construtor dele. Quando a gente só declara uma referência, como `Aluno a;`, a gente cria apenas uma variável que pode apontar para um aluno, mas ainda não existe aluno nenhum (ela fica `null`). O objeto só passa a existir quando usamos o `new`, por exemplo `a = new Aluno("Ana");`. Então declarar é criar o "controle remoto" e criar o objeto é criar a "TV".

4. Explique encapsulamento. Por que evitar atributos públicos sem necessidade?

Encapsulamento é esconder os dados de dentro da classe, deixando os atributos `private` e só permitindo acessar por métodos, como os getters. Isso evita atributos públicos porque qualquer parte do código poderia mudar o valor sem controle, até para um valor errado, como um preço negativo. Com o encapsulamento a classe se protege e valida os dados. Na classe `Curso`, por exemplo, o preço é privado e o construtor não aceita preço menor ou igual a zero.

5. Qual é a finalidade do construtor? Explique o uso de this.

O construtor é um método especial que roda quando o objeto é criado. Ele tem o mesmo nome da classe, não tem tipo de retorno e serve para dar os valores iniciais e validar os dados, para o objeto já nascer certo. O `this` é uma referência ao próprio objeto. A gente usa principalmente quando o parâmetro tem o mesmo nome do atributo, como em `this.titulo = titulo;`, para o Java saber que o da esquerda é o atributo da classe.

Bloco B - Relacionamentos e herança

6. Diferencie associação, agregação e composição; exemplifique.

Na **associação** uma classe se relaciona com outra, mas as duas existem independentes. Exemplo: Aluno e Curso, o curso continua existindo se o aluno sair. Na **agregação** existe um todo e partes, mas as partes podem existir sozinhas. Exemplo: um Departamento com Professores, se o departamento fechar os professores continuam existindo. Na **composição** a parte depende do todo e some junto com ele. Exemplo: um Pedido e os seus itens, se o pedido for apagado os itens não fazem mais sentido. No nosso sistema, Aluno e Curso é associação.

7. O que é ArrayList e quando preferi-lo a um vetor de tamanho fixo?

O `ArrayList` é uma lista do Java (`java.util`) que cresce e diminui sozinha conforme a gente adiciona ou remove elementos, e já tem métodos prontos como `add`, `remove`, `get` e `size`. Ele é melhor que o vetor quando não sabemos quantos elementos vão ser guardados, como os cursos de um aluno, que podem ser 1 ou 10. O vetor comum tem tamanho fixo definido na criação, então só vale a pena quando já sabemos o tamanho exato.

8. Diferencie superclasse e subclasse. Explique extends e super.

A superclasse é a classe mais geral, de onde se herda. A subclasse é a classe mais específica, que herda da superclasse e pode ter coisas a mais ou mudar o que herdou. O `extends` é a palavra que faz a herança, como em `class CriadorTexto extends CriadorComprovante`. O `super` é usado para acessar a superclasse, seja para chamar o construtor dela, com `super(...)`, ou um método dela, com `super.metodo()`.

9. Diferencie sobrecarga e sobrescrita de métodos; dê exemplos.

Na **sobrecarga** temos métodos com o mesmo nome na mesma classe, mas com parâmetros diferentes. Exemplo: `somar(int a, int b)` e `somar(double a, double b)`. Na **sobrescrita** a subclasse reescreve um método que veio da superclasse, mantendo o mesmo nome e os mesmos parâmetros, e usa `@Override`. Exemplo: `CriadorTexto` sobrescreve o método `criar()` da classe `CriadorComprovante`.

10. O que é polimorfismo? Como uma referência de superclasse aponta para objetos de subclasses?

Polimorfismo quer dizer "muitas formas": o mesmo método pode ter comportamentos diferentes dependendo do objeto que o executa. Isso é possível porque uma variável do tipo da superclasse pode guardar um objeto de qualquer subclasse. Na hora de executar, o Java usa o método do objeto real. No exemplo abaixo, a variável é do tipo da superclasse, mas o objeto é do tipo `CriadorResumido`, então o comprovante sai no formato resumido.

```
CriadorComprovante c = new CriadorResumido();
String s = c.emitir(aluno, curso, 180.0); // usa ComprovanteResumido
```

Bloco C - Interfaces e projeto

11. O que é interface? Diferencie implements e extends nos exemplos estudados.

Interface é um contrato: ela só diz quais métodos a classe tem que ter, sem dizer como fazer. Exemplo: a interface `Desconto` tem o método `aplicar(double valor)`. O `implements` é usado quando uma classe assina esse contrato, como `class DescontoEstudante implements Desconto`. Já o `extends` é usado na herança entre classes, como `CriadorTexto extends CriadorComprovante`. Uma classe só pode estender uma classe, mas pode implementar várias interfaces.

12. Compare herança e composição. Quando a composição é preferível?

Herança é a relação "é um" (um aluno é uma pessoa) e deixa as classes bem ligadas uma na outra. Composição é a relação "tem um": a classe guarda outro objeto dentro dela e usa ele. A composição é melhor quando queremos mais flexibilidade, por exemplo poder trocar o comportamento sem mexer na classe, ou quando não existe uma relação "é um" de verdade. No projeto, o `ServicoMatricula` tem um `Desconto`, ele não herda de desconto.

13. O que é baixo acoplamento? Como a injeção por construtor ajuda?

Baixo acoplamento é quando as classes dependem pouco umas das outras, de preferência só de interfaces. Assim, mudar uma classe não quebra as outras e fica mais fácil testar e trocar peças. A injeção por construtor ajuda porque a classe recebe o que precisa pelo construtor, em vez de criar com `new` lá dentro. O `ServicoMatricula` recebe um `Desconto` e um `RegistradorPagamento` pelo construtor, então funciona com qualquer implementação deles.

14. Explique SRP, OCP e DIP com exemplos simples.

SRP (responsabilidade única): cada classe deve ter um só motivo para mudar. Exemplo: `Curso` cuida dos dados do curso e o cálculo do desconto fica em outra classe. **OCP (aberto/fechado):** a classe deve estar aberta para extensão e fechada para modificação. Exemplo: para criar um novo desconto, basta criar uma nova classe que implementa `Desconto`, sem mexer no `ServicoMatricula`. **DIP (inversão de dependência):** as classes devem depender de abstrações e não de classes concretas. Exemplo: o `ServicoMatricula` depende da interface `RegistradorPagamento` e não direto do `SistemaLegado`.

Bloco D - Padrões de projeto

15. O que é um padrão de projeto? Por que evitar aplicá-lo indiscriminadamente?

Padrão de projeto é uma solução já conhecida e testada para um problema que aparece sempre no desenvolvimento de software. Não é um código pronto, é uma ideia de como organizar as classes. Não devemos usar em tudo porque, se o problema não existir, o padrão só deixa o código mais complicado, com classes a mais sem necessidade. A gente deve usar quando ele realmente resolve algo.

16. Explique Singleton e a finalidade do construtor privado e de getInstancia().

O Singleton garante que a classe tenha só uma instância no programa inteiro e dá um jeito de acessar ela de qualquer lugar. O construtor privado serve para ninguém conseguir criar objetos com `new` fora da classe. O método `getInstancia()` é quem entrega a única instância que já foi criada. Na classe `Configuracao`, a instância é criada uma vez só e todo mundo que chama `getInstancia()` recebe o mesmo objeto.

17. Explique Factory Method e o papel dos criadores abstratos e concretos.

O Factory Method coloca a criação do objeto num método, e deixa as subclasses decidirem qual objeto concreto vai ser criado. O criador abstrato, `CriadorComprovante`, tem o método abstrato `criar()` e o método `emitir()`, que usa o comprovante sem saber de que tipo ele é. Os criadores concretos, `CriadorTexto` e `CriadorResumido`, sobrescrevem o `criar()` e devolvem o `ComprovanteTexto` ou o `ComprovanteResumido`. Se quiser um formato novo, é só criar mais uma subclasse.

18. Como Strategy organiza descontos comuns, estudantis e promocionais?

No Strategy, cada forma de fazer uma coisa vira uma classe separada que segue a mesma interface. Aqui a interface é `Desconto`, e cada tipo de desconto é uma classe: `SemDesconto` para o comum, `DescontoEstudante` para o estudantil e uma `DescontoPromocional` para o promocional. O serviço recebe o desconto escolhido e chama `aplicar()` sem precisar saber qual é. Assim não precisa ficar usando vários `if/else`, e para trocar de política é só passar outro objeto.

19. Como Observer comunica um pedido confirmado a estoque, financeiro e auditoria?

No Observer existe um sujeito, que é quem tem o evento, e vários observadores, que querem ser avisados. Estoque, financeiro e auditoria seriam os observadores: cada um implementa a mesma interface com o método `atualizar()` e se inscreve no serviço. Quando o pedido é confirmado, o serviço percorre a lista de inscritos e chama `atualizar()` em cada um. Dessa forma o serviço não precisa conhecer cada setor, e dá para adicionar novos observadores sem mudar o serviço. No projeto isso acontece com `AvisoEmail` e `Auditoria` depois da matrícula.

20. Diferencie Adapter e Facade; dê um caso de uso de cada um.

O **Adapter** serve para fazer duas coisas incompatíveis funcionarem juntas, traduzindo uma interface para a outra. Exemplo: o `AdaptadorPagamento` recebe o valor em reais e converte para centavos, porque o `SistemaLegado` só aceita centavos. O **Facade** serve para simplificar o uso de várias classes através de uma única classe mais fácil. Exemplo: a `FachadaAcademica` faz a matrícula e emite o comprovante num método só, e o `Main` não precisa se preocupar com os detalhes. Resumindo: o Adapter adapta e o Facade simplifica.

Parte II - Perguntas de análise da implementação (Etapa 8)

1. Qual classe implementa Singleton e quais instruções controlam a instância?

É a classe `Configuracao`. Três coisas controlam a instância: o atributo `private static final Configuracao INSTANCIA = new Configuracao();`, que cria o único objeto, o construtor privado `private Configuracao() {}`, que impede criar outros, e o método `getInstancia()`, que devolve esse objeto.

2. Onde aparecem encapsulamento e validação?

Nas classes `Curso` e `Aluno`. Os atributos são `private final` e só podem ser lidos pelos getters. Os construtores validam os dados e lançam `IllegalArgumentException` se o nome ou título for vazio ou nulo, ou se o preço for menor ou igual a zero. O método `adicionarCurso()` também valida e não aceita curso nulo.

3. Qual é a relação entre Aluno e Curso?

É uma associação. O `Aluno` guarda uma lista de `Curso`, mas o curso existe por conta própria. Se o aluno deixar de existir, o curso não precisa ser destruído, por isso não é composição.

4. Onde foi utilizada uma coleção e com qual objetivo?

Em dois lugares. Na classe `Aluno`, uma `ArrayList<Curso>` guarda os cursos do aluno, já que a quantidade pode variar. Na classe `ServicoMatricula`, uma lista de `ObservadorMatricula` guarda os observadores inscritos, para avisar todos depois da matrícula.

5. Qual é o papel da interface Desconto?

Ela é o contrato do padrão Strategy. Define o método `aplicar(double valor)`, e assim o `ServicoMatricula` consegue usar qualquer desconto sem depender de uma classe concreta. As classes que seguem esse contrato são `SemDesconto` e `DescontoEstudante`.

6. Como acrescentar uma política de 20% sem alterar o serviço?

É só criar uma nova classe que implementa `Desconto` e passar ela no construtor do `ServicoMatricula`. O serviço não precisa ser alterado, o que segue o princípio OCP.

```
public class DescontoPromocional implements Desconto {
    @Override
    public double aplicar(double valor) { return valor * 0.80; }
}

// no Main:
new ServicoMatricula(new DescontoPromocional(), new AdaptadorPagamento(new SistemaLegado()));
```

7. Quem produz o evento e quem recebe as notificações?

Quem produz o evento é o `ServicoMatricula`, no método `matricular()`, quando percorre a lista e chama `atualizar()`. Quem recebe as notificações são os observadores inscritos, que são o `AvisoEmail` e a `Auditoria`.

8. Por que o adaptador converte reais em centavos?

Porque o `SistemaLegado` só aceita centavos em número inteiro, no método `registrarEmCentavos(int)`, e o nosso sistema trabalha em reais. O `AdaptadorPagamento` faz a conta `Math.round(reais * 100)` para as duas partes se entenderem, sem precisar mexer no sistema antigo. Por exemplo, R$ 180,00 vira 18000 centavos.

9. Qual método representa Factory Method?

É o método `criar()`. Ele é abstrato na classe `CriadorComprovante` e é sobrescrito em `CriadorTexto` e `CriadorResumido`, que devolvem o tipo de comprovante certo.

10. O que FachadaAcademica simplifica para Main?

Ela esconde os passos do caso de uso: aplicar desconto, registrar o pagamento, matricular, avisar os observadores e gerar o comprovante. Com isso o `Main` só precisa chamar `matricularEEmitir(aluno, curso)`.

Saída esperada da execução

```
Sistema Acadêmico
Legado registrou: 18000 centavos
E-mail: Ana matriculado em Java
Auditoria: Ana matriculado em Java
Matrícula: Ana / Java / R$ 180.0
Cursos do aluno: 1
```

Nota: o código usa `double` para valores monetários para fins didáticos; em sistemas financeiros reais o recomendado é `BigDecimal`.